



MICRO SERVIÇOS E SISTEMAS ESCALÁVEIS

Aula 1 — Do Monolito aos Microserviços

Uma introdução ao porquê, ao quando e ao como da arquitetura de microserviços

Prof. Renan de Carvalho Oliveira

Curso de Sistemas de Informação · 7º período · Disciplina SIS440

Duração: 2 horas-aula

O que vamos ver hoje

01

Contexto

Por que o jeito de construir software mudou nos últimos anos

03

Fundamentos

Conceitos-base de sistemas distribuídos que sustentam tudo o resto

05

Trade-offs

Benefícios e desafios reais dessa arquitetura

02

O problema

O que é um monolito e por que ele "trava" o crescimento

04

A solução

O que são microsserviços, na prática

06

A disciplina

Panorama dos 8 módulos, metodologia e avaliação

O software mudou de escala

Aplicações que antes atendiam milhares de usuários dentro de uma empresa hoje precisam atender milhões de pessoas, o tempo todo, em qualquer lugar do mundo. Times cresceram, prazos encurtaram e o negócio passou a exigir entregas contínuas, não mais um grande lançamento por ano.

Esse novo cenário expõe os limites de arquiteturas pensadas para um contexto mais simples — e é exatamente esse limite que vamos entender hoje, antes de chegar à solução.

Entrega contínua

Deploys que antes levavam semanas passam a acontecer várias vezes ao dia.

Times autônomos

Squads que precisam evoluir suas partes do sistema sem esperar os outros.

Escala sob demanda

Picos de uso exigem crescer (e encolher) rapidamente, só onde é preciso.

O ponto de partida: a arquitetura monolítica

Um monólito é uma aplicação construída e implantada como uma única unidade: todo o código-fonte, toda a lógica de negócio e, geralmente, um único banco de dados. Não é um erro de arquitetura — é, inclusive, a escolha certa no início da vida de um sistema.

Vantagens quando a aplicação ainda é pequena:

- ✓ Simples de desenvolver e depurar
- ✓ Fácil de testar de ponta a ponta
- ✓ Deploy é copiar um único artefato
- ✓ Escalar é rodar várias cópias iguais



Quando o próprio sucesso vira um problema

À medida que a aplicação e o time crescem, os mesmos pontos fortes do monólito começam a travar a evolução do sistema.



Complexidade sufocante

O código cresce até que ninguém mais entende o sistema por completo — cada mudança vira um risco.



Desenvolvimento lento

IDE, build e testes ficam cada vez mais pesados; o ciclo editar → rodar → testar não para de crescer.



Deploy longo e arriscado

Colocar em produção vira um evento raro, manual e tenso — geralmente feito à noite, "torcendo para dar certo".



Escala não uniforme

Módulos com necessidades diferentes de CPU/memória são forçados a escalar juntos, sem separação.



Falha em cascata

Um vazamento de memória em um módulo pode derrubar a aplicação inteira, não apenas a parte afetada.



Tecnologia presa no tempo

Trocar de framework ou linguagem exige reescrever tudo de uma vez — algo caro e arriscado demais.



PARA DISCUTIR EM SALA

Vocês já sentiram algum desses sintomas em algum sistema — na faculdade, em estágio ou em projeto pessoal?

Pensem em: tempo de build, medo de mexer em algum módulo, dificuldade de escalar só uma parte do sistema.

Antes dos microsserviços: sistemas distribuídos

Microsserviços são, na essência, uma forma de sistema distribuído. Quatro propriedades vão guiar toda a disciplina:



Transparência

O sistema se comporta, para quem usa, como se fosse uma coisa só — mesmo estando espalhado em várias máquinas.



Escalabilidade

Capacidade de crescer (usuários, dados, requisições) sem redesenhar o sistema do zero.



Openness (abertura)

Componentes seguem padrões e interfaces bem definidas, permitindo integrar e substituir peças com facilidade.



Confiabilidade

O sistema continua funcionando (ou falha de forma controlada) mesmo quando uma parte apresenta problema.

Três formas de escalar um sistema

O "cubo de escala" (Abbott & Fisher) organiza as estratégias de escalabilidade em três eixos:

EIXO X

Clonagem

Rodar várias instâncias idênticas atrás de um balanceador de carga. Aumenta capacidade e disponibilidade.

EIXO Y

Decomposição funcional

Dividir o sistema por funcionalidade (pedidos, pagamentos, entregas...). É o caminho para os microsserviços.

EIXO Z

Particionamento

Rodar instâncias iguais, cada uma responsável por uma fatia dos dados (ex.: por região ou por cliente).

O que são, de fato, microsserviços?

Um estilo arquitetural que decompõe uma aplicação em um conjunto de serviços pequenos, independentes e organizados em torno de funcionalidades de negócio.



Serviço = unidade de modularidade

Cada serviço tem uma API bem definida; ninguém acessa seu banco ou suas classes internas por fora dela.



Um banco de dados por serviço

Cada serviço é dono dos seus próprios dados — não existe um banco único compartilhado por todos.



Deploy independente

Um serviço pode ser implantado sem esperar (nem travar) os demais.



Times autônomos

Cada equipe pode desenvolver, testar e escalar seus serviços de forma independente.

Monólito, SOA e Microserviços lado a lado

	Monólito	SOA	Microserviços
Unidade de deploy	Uma só, para tudo	Poucos serviços grandes	Muitos serviços pequenos
Comunicação	Chamada de método local	Barramento (ESB), protocolos pesados	REST/gRPC ou mensageria leve
Dados	Um banco compartilhado	Modelo de dados global	Um banco por serviço
Times	Um time grande ou vários no mesmo código	Times por domínio de negócio	Times pequenos e autônomos

Benefícios da arquitetura de microsserviços

-  **Entrega contínua** Serviços pequenos são mais fáceis de testar e implantar — deploys frequentes deixam de ser um evento arriscado.
-  **Escalabilidade independente** Cada serviço escala de acordo com sua própria demanda, sem carregar o resto do sistema junto.
-  **Isolamento de falhas** Um problema em um serviço não derruba a aplicação inteira.
-  **Liberdade tecnológica** Cada serviço pode usar a linguagem/framework mais adequado ao seu problema.
-  **Times autônomos** Squads pequenos ganham independência para desenvolver e lançar suas partes do sistema.

Nenhuma arquitetura é bala de prata

Ganhar essas vantagens tem um preço: complexidade que passa a existir em outros lugares.



Encontrar os limites certos

Não existe fórmula pronta para dividir o sistema em serviços; decompor mal gera um "monólito distribuído".



Complexidade distribuída

Chamadas de rede substituem chamadas de método: é preciso lidar com latência e falhas parciais.



Consistência de dados

Sem um banco único, transações e consultas que cruzam serviços exigem técnicas específicas (sagas, composição de API).



Coordenação de deploys

Funcionalidades que atravessam vários serviços exigem planejar a ordem de implantação entre times.



Quando adotar?

No início de um produto, o monólito costuma ser a escolha certa; microsserviços resolvem problemas de escala e complexidade que ainda não existem numa aplicação nova.

Para onde vamos: os 8 módulos



Hoje cobrimos a introdução ao Módulo 1 — Fundamentos. A partir da próxima aula, entramos a fundo em cada um dos 8 módulos.

Como vamos trabalhar e como vocês serão avaliados

METODOLOGIA



Aulas expositivas com exemplos práticos



Estudos de caso de sistemas reais



Exercícios de fixação em sala



Discussões e debates sobre arquitetura



Análise de problemas do mundo real

AVALIAÇÃO (por bimestre)

Avaliação Online (AVA)	20 pts
Avaliação Teórica	30 pts
Listas de atividades / exercícios	≈ 35 pts
Projeto Final / Simulado (2º bim.)	≈ 15 pts

Aprovação direta com MS ≥ 70 · entre 40 e 70 há Prova Final



PARA A PRÓXIMA AULA

Fechando o Módulo 1: Fundamentos de Sistemas Distribuídos

- ➔ Aprofundar transparência, escalabilidade, openness e confiabilidade com exemplos práticos
- ➔ Trade-offs e desafios de projetar sistemas distribuídos
- ➔ Leitura recomendada: capítulo 1 de Microservices Patterns (Richardson) e capítulo 1 de Sistemas Distribuídos (Tanenbaum & Van Steen)

Bibliografia básica desta aula: NEWMAN, S. Building Microservices; RICHARDSON, C. Microservices Patterns; TANENBAUM, A.; VAN STEEN, M. Sistemas Distribuídos.